Access Control

2026-11-04

Table of contents

1 Access Control **1**
- 1.1 Authentication Factors . . . 2
- 1.2 Guiding Principles . . . 3
- 1.3 Access Control Models . . . 3

2 Port Knocking **4**

3 Single Packet Authorization **5**
- 3.1 SPA and Zero Trust . . . 6
- 3.2 A Note on Firewalls: iptables and nftables . . . 6

4 Chroot Jails **7**
- 4.1 Limitations . . . 7

1 Access Control

Access control refers to the methods and policies used to restrict and manage who can view or use resources. It's about ensuring that only authorized individuals can access certain data, systems, or functions and preventing unauthorized access.

The key concepts of access control are identification, authentication, authorization, and accountability.

- Identification is making a claim about one's identity (*who are you?*). Example: typing a username.
- Authentication is confirming that claim (*are you who you say you are?*). Example: entering the password for that username.

- Authorization is specifying the access rights an authenticated identity has to resources (*are you allowed to access this?*). Example: file permissions.
- Accountability is the acknowledgement and assumption of responsibility for actions. It relies on logging and auditing so that actions can be traced back to an identity.

Access control needs to be enforced at various levels for different systems (e.g., the operating system, applications, databases, and the network).

1.1 Authentication Factors

Authentication is typically based on one or more of the following factors:

- Something you **know**: a password, PIN, or answer to a security question.
- Something you **have**: a key fob, smart card, hardware token (dongle), or phone running an authenticator app.
- Something you **are**: physical biometrics (fingerprint, face, iris) or behavioral biometrics (keystroke dynamics, gait).

Multi-factor authentication (MFA) combines factors from *different* categories. Two passwords is still a single factor.

Not all factors are equally strong:

- **SMS codes and push approvals** are better than a password alone, but they can be phished (a fake login page asks for the code) or abused (an attacker sends repeated push requests until the user taps "approve", known as *MFA fatigue*).
- **Phishing-resistant MFA**, such as FIDO2 security keys and passkeys, uses public-key cryptography tied to the real website's domain. A fake site cannot receive a usable response, so these resist phishing in a way that typed codes do not.

Password guidance has also changed. Current NIST guidance (SP 800-63B) favors long passwords and passphrases over composition rules (forced symbols, numbers, and uppercase letters), recommends checking new passwords against lists of known breached passwords, and no longer recommends forcing users to change passwords on a fixed schedule. Passwords should be changed when there is evidence of compromise.

1.2 Guiding Principles

- **Least privilege**: give every user and process only the access it needs to do its job, and nothing more.
- **Separation of duties**: split sensitive tasks so that no single person can complete them alone (e.g., the person who requests a payment is not the one who approves it).
- **Defense in depth**: layer multiple controls so that the failure of one does not mean the failure of all. The tools in this unit (SPA and chroot jails) are extra layers, not replacements for authentication.

1.3 Access Control Models

There are four major access control models:

1. Discretionary Access Control (DAC) - Every object in a system has an owner who can grant access to other users at their discretion. Standard Linux file permissions (`rwx` for owner, group, and others, changed with `chmod` and `chown`) are an example of DAC. Access control lists (ACLs), managed on Linux with `setfacl` and `getfacl`, extend DAC by letting an owner grant permissions to specific additional users and groups.
2. Mandatory Access Control (MAC) - Access is decided by a system-wide policy that users cannot override, even for objects they own. Subjects and objects are given labels, and the system compares them on every access. Clearance levels (e.g., Confidential, Secret, Top Secret) are the classic example, as formalized in the Bell-LaPadula model, which is common in governmental and military contexts. On Linux, SELinux and AppArmor are MAC systems. For example, the `fwknop-apparmor-profile` package you will see in Lab 1.1 is an AppArmor policy that limits what the fwknop daemon may access, even though it runs as root.
3. Role-Based Access Control (RBAC) - Access is granted to roles based on the functions of that role, and users are assigned to roles. For example, the instructors at a university have a different role than the students and can access information concerning individual students in ways that students cannot.
4. Attribute-Based Access Control (ABAC) - Access decisions are made by evaluating attributes of the subject (e.g., department, clearance), the object (e.g., classification, owner), and the environment (e.g., time of day, location of the request). This model offers the most fine-grained control over who gets access to what.

2 Port Knocking

In networking, a port is a numbered endpoint on a machine that a particular service listens on. Each port is like a "channel" where a specific type of traffic is handled. To refer to a port, you typically write the ip address followed by a colon, then the port. For example: `138.47.123.123:8000` refers to port 8000. Ports are numbered from 0 to 65535 and fall into three ranges:

- **Well-known ports (0-1023)**: assigned to common services such as SSH and HTTP. On Linux, only root can listen on these by default.
- **Registered ports (1024-49151)**: assigned to specific applications by IANA (e.g., 3306 for MySQL, 3389 for Remote Desktop).
- **Dynamic or ephemeral ports (49152-65535)**: picked temporarily by a client for the client's end of a connection.

Some ports are reserved for certain usage. For example, if you're using your computer to host a website, you will use port 443 to deliver content over HTTPS and port 80 to deliver unsecured HTTP content.

Here is a list of some of the well-known ports and their usages:

- 20 - FTP (File transfer data)
- 21 - FTP (Control commands for file transfer)
- 22 - SSH (Secure Shell access)
- 25 - SMTP (Email)
- 53 - DNS (Name resolution)
- 80 - HTTP (Regular web traffic, not secure)
- 143 - IMAP (Email)
- 443 - HTTPS (Web traffic, secure)

Ports are the way remote users get into a machine. If a port is closed (or filtered by a firewall), no one can connect to the service behind it. Security-wise, it would be great to keep all of the ports closed; however, if we want to remotely access a computer, this is simply not possible.

A port scanner such as `nmap` reports a port in one of three main states, based on how the target responds:

- **Open**: a service is listening and accepts the connection.
- **Closed**: nothing is listening, and the host replies to say so (a TCP reset, or for UDP an ICMP "port unreachable" message). The scanner learns the host exists.
- **Filtered**: there is no reply at all, because a firewall silently dropped the probe. The scanner cannot tell whether a service is behind it.

A firewall rule that *drops* packets produces **filtered**, which reveals the least to an attacker. You will see this in Lab 1.2.

One way around this is port knocking. With port knocking, the service's port stays closed by the firewall. A daemon on the server watches for connection attempts to a secret *sequence* of other (closed) ports. When it sees the correct sequence (the "knock") from a client, it temporarily opens the real port for that client's ip address only. To everyone else, including port scanners, the service appears not to exist.

Port knocking has several forms:

- Knocking on several ports in a timed sequence.
- Knocking on a single port with a symmetric key.
- Knocking with a single, encrypted and authenticated packet (single packet authorization).

Note that a plain knock sequence is basically security through obscurity, which should not be the security we count on. Anyone who can observe the network traffic can see the sequence and replay it.

3 Single Packet Authorization

Single packet authorization (SPA) is a network security technique used to control access to available services. SPA works through the following simple process:

1. The client sends a single SPA packet to the server. The packet's contents are encrypted and protected by an HMAC.
2. The server checks the validity of the SPA packet. This includes verifying the HMAC, decrypting the packet, checking the source ip address, checking a timestamp, and checking that the packet has not been seen before.
3. If the SPA packet contains valid information, then the server adds a temporary firewall rule granting access to the appropriate ports (for example, SSH) for the client's ip address only.
4. If invalid, the server silently drops the packet and no feedback is given.

SPA is significantly more secure than a plain knock sequence:

- **No replay**: each SPA packet contains random data and a timestamp, and the server remembers packets it has already accepted. A captured packet cannot simply be re-sent by an attacker.

- **Real cryptography**: rather than relying on a secret sequence that can be observed, the packet is protected with keys shared between client and server. In fwknop, the payload is encrypted with Rijndael (AES) and authenticated with an HMAC (SHA-256 by default). GPG keys can be used instead of a shared encryption key.
- **Nothing to scan**: the fwknop server (`fwknopd`) does not listen on a port. It passively sniffs packets (UDP port 62201 by default), so a port scan of the server shows nothing.
- **One packet**: a single packet is harder to notice and is not affected by packets arriving out of order.

SPA is an extra layer of defense in depth. Once the port is open, the client must still authenticate to the service itself (e.g., with an SSH password or key).

3.1 SPA and Zero Trust

SPA is a building block of the *Software-Defined Perimeter* (SDP), an architecture published by the Cloud Security Alliance that applies *Zero Trust* ideas to network access. Zero Trust means no device is trusted just because it is on the "inside" of a network; every request must be authenticated and authorized. In an SDP, services are invisible ("dark") to everyone, and a client must prove who it is, often with an SPA packet, before it can even see that a service exists.

3.2 A Note on Firewalls: iptables and nftables

The labs use the `iptables` command, which is still widely taught and documented. On current Ubuntu, Debian, and Mint releases, however, `iptables` is a compatibility front end (`iptables-nft`) for the newer *nftables* framework in the Linux kernel. The commands work the same, but the rules are stored as nftables rules. You can see the same rules in nftables form with:

```
sudo nft list ruleset
```

You can check which back end your system uses with `iptables --version`; it prints `(nf_tables)` or `(legacy)`.

4 Chroot Jails

Every process on Linux has a *root directory*, which is the directory it sees as `/`. Normally this is the real root of the file system. The `chroot` command (and the underlying `chroot()` system call) changes the root directory of a process and all of its children to some other directory. That directory is called a chroot jail.

Inside the jail, a process cannot name any file outside of the jail, since `/` now refers to the jail directory and `cd ..` from `/` goes nowhere. This means that everything the process needs, including the programs it runs, their shared libraries, and configuration files, must be copied into the jail.

Common uses for chroot jails include:

- Limiting what FTP, SFTP, or SSH users can see and do on a server (OpenSSH supports this directly with the `ChrootDirectory` option).
- Creating a clean, isolated environment for building or testing software.
- Repairing a broken system by booting from a live USB and chrooting into the installed system.

4.1 Limitations

A chroot jail is **not** a complete security boundary:

- **Root can escape**: a process running as root inside a jail can call `chroot()` again into a subdirectory and then walk back out with `..`, escaping the jail. This is why jailed users should never have root access or `sudo` inside the jail. For the same reason, never copy *setuid* programs (programs that run with their owner's privileges, such as `sudo`, `su`, or `passwd`) into a jail, since they give a path to root.
- **Only the file system is isolated**: processes in the jail still share the same process list, network, users, and kernel with the rest of the system.

Modern Linux addresses these limitations with *namespaces* (isolating processes, networking, mounts, users, and more) and *cgroups* (limiting resources). Container tools such as Docker and Podman are built on these features, and can be thought of as the grown-up successors of the chroot jail.